

AI Audit Report — Biểu mẫu 5 phần cho mỗi artifact

Phụ lục bắt buộc cho mọi bài tập có sử dụng AI (HW#01–HW#06, và Seminar).

Phỏng theo Med Kharbach, PhD (2026) — AI Use Policy Templates for Higher Education. CC BY-NC-SA 4.0. Bản chuyển thể này được soạn cho môn CS423 / CSC15003 Kiểm thử Phần mềm tại FIT@HCMUS.

1. Thông tin sinh viên

Trường thông tin	Giá trị
Họ và tên (viết in):	TRƯƠNG THÀNH ĐẠT
MSSV:	23127344
Lớp / Khóa:	23KTPM3 — Khóa K23
Mã bài tập (ví dụ: HW#00, HW#02):	HW#05 — Kiểm thử Hiệu năng (HW05-AI)
Ngày làm bài:	2026-08-12 → 2026-08-14
Công cụ AI đã dùng:	Claude Opus 5 (Claude Code, tiện ích mở rộng VS Code) — công cụ AI duy nhất dùng trong bài này

x

Có sử dụng AI hay không:

Có

Không

2. Hướng dẫn (đọc trước khi điền)

- Thêm một dòng cho mỗi artifact do AI sinh ra (test case, script, checklist, đặc tả OpenAPI, JMeter plan, v.v.).
- Dán **nguyên văn** prompt — KHÔNG diễn giải lại.
- Dán **nguyên văn** output của AI (hoặc chèn ảnh chụp màn hình có chú thích trong báo cáo).
- Gán nhãn kết luận: VALID / INVALID / INCOMPLETE.
- Phần lập luận phải dẫn chiếu tới một slide bài giảng, một mục ISTQB, hoặc một RFC kỹ thuật.
- Trình bày artifact đã sửa, có làm nổi bật phần thay đổi.
- Các dòng ví dụ được in nghiêng — phải thay thế trước khi nộp.

3. Bảng kiểm toán — mỗi artifact một dòng

(1) Prompt + Công cụ	(2) Output của AI	(3) Kết luận	(4) Lập luận (ISTQB)	(5) Phần sinh viên sửa
Artifact #1 — Khung báo cáo chính (Main_Report.md) Công cụ: Claude Opus 5 (Claude Code, tiện ích VS Code) Thời điểm: 22:34 12/08/2026 Prompt (nguyên văn): "@hw5/2026.HW05.Performance Testing_En.pdf read the homework and than write for me the Main_Report.md and I will fill in the information."	Đọc bản đặc tả 9 trang và tạo ra hw5/Main_Report.md: một bộ khung theo từng mục (§1 phạm vi / lựa chọn endpoint; §2 phần cứng + môi trường SUT; §3 Task 1 — chuỗi thiết kế cùng AI, luồng E2E, bảng correlation và assertion, danh mục CSV, ma trận tham số Load/Stress/Spike, ba report view khác nhau, bảng rà soát của con người, kết quả thực thi, reset lockout, ngưỡng endurance, video, danh sách lỗi; §4 Task 2 — truy tìm điểm hiểu sai và bảng phân loại khả thi / ảo giác; §5 Task 3 — mô hình CPT kèm lưu đồ Mermaid và bảng đánh đổi; §6 Agent Skill; §7 Phê bình AI; §8 nhật ký commit; §9 danh sách kiểm tra sản phẩm nộp; §10 tự đánh giá theo đúng trọng số 20/20/20/10/10/10; §11 tài liệu tham khảo; phụ lục A–B). Mọi trường số liệu đo được đều để dạng <code><placeholder></code>. AI suy ra MSSV 23127344 từ tên nhánh git và đã đánh dấu để tôi xác nhận lại.	INCOMPLETE	Cấu trúc tuân thủ đúng §6 và §14 của đề bài, nhưng artifact này là một biểu mẫu tài liệu, không phải sản phẩm đã kiểm chứng — toàn bộ số liệu còn là placeholder. ISTQB FL §1.4.3 (giám sát và kiểm soát kiểm thử phải dựa trên kết quả đo thực tế) và §5.3 (báo cáo tiến độ kiểm thử phải dựa trên dữ liệu thu thập được, không phải dữ liệu dự kiến). Ngoài ra AI không thể tự kiểm chứng các endpoint thật của SUT, nên §1 của báo cáo chưa được xác thực: theo ISTQB FL §3.2.1, việc kiểm chứng test basis thuộc trách nhiệm người rà soát.	<Mô tả phần bạn đã sửa: xác nhận / chỉnh lại MSSV; thay các dòng endpoint ở §1 bằng route thật đọc từ repo eshop-sut; xóa những mục không áp dụng; tự viết toàn bộ phần phê bình §7; điền mọi placeholder từ file <code>.jtl</code> thô.>

(1) Prompt + Công cụ	(2) Output của AI	(3) Kết luận	(4) Lập luận (ISTQB)	(5) Phần sinh viên sửa
Artifact #2 — Đề xuất luồng nghiệp vụ từ endpoint suy đoán (kiểm tra trùng lặp) Công cụ: Claude Opus 5 (Claude Code, tiện ích VS Code) Thời điểm: 22:34 12/08/2026 Prompt (nguyên văn): "Those pictures all the api my team test. do you have propose any api for me to finnish the hw" <i>(đính kèm: 4 ảnh chụp báng endpoint của các bạn cùng nhóm)</i>	Đọc bốn ảnh chụp, liệt kê các endpoint mà nhóm đã nhận, rồi đề xuất "hành trình hồ sơ cá nhân + lịch sử đơn hàng": 1) <code>POST /api/login</code> → 2) <code>GET /api/profile</code> (hoặc <code>/api/users/me</code>) → 3) <code>GET /api/orders/:id</code> → 4) <code>PUT /api/profile</code> → 5) <code>POST /api/orders</code> hoặc một bước chuyển trạng thái đơn hàng. Lập luận rằng đề bài cấm trùng luồng nghiệp vụ chứ không cấm trùng endpoint, nên việc dùng chung <code>POST /api/login</code> là chấp nhận được, còn các bước read-heavy và transactional thì bắt buộc phải khác nhau. Có đề xuất thêm một luồng dự phòng dùng coupon/giỏ hàng. AI tự đánh dấu output của mình là chưa kiểm chứng, nói rõ các đường dẫn chỉ là "suy đoán hợp lý từ FR-04/FR-11, không hơn" và rằng nó chưa đọc repository của SUT.	INVALID	Hai đường dẫn được đề xuất là <code>GET /api/profile</code> và <code>PUT /api/profile</code> không tồn tại trong SUT. Endpoint thật là <code>GET /api/users/me</code> và <code>PUT /api/users/me</code> (API spec §2.1, §2.2). AI đã sinh tên route bằng cách khớp mẫu từ tiêu đề của FR thay vì đọc test basis. Theo ISTQB FL §3.2.1, khiếm khuyết trong test basis phải được phát hiện trước khi bắt đầu thiết kế kiểm thử — nếu dựng file JMX từ các đường dẫn này thì kết quả sẽ là một bộ test toàn lỗi 404, dễ bị hiểu nhầm thành lỗi của SUT. ISTQB FL §1.4.2 cũng áp dụng: phân tích kiểm thử phải bắt đầu từ một test basis đã được khảo sát, không phải từ giả định.	Cung cấp tài liệu chính thức <code>api_spec.md</code> cho AI và yêu cầu mọi endpoint đều phải kèm số mục tham chiếu trong đặc tả. Đường dẫn đã được sửa thành <code>/api/users/me</code>. Giữ nhãn INVALID thay vì INCOMPLETE: artifact này không chỉ sơ sai mà còn gọi tên những tài nguyên không tồn tại, và không có cách chỉnh sửa nào giữ lại được bản gốc. Xem Artifact #3 cho thiết kế đã sửa.

(1) Prompt + Công cụ	(2) Output của AI	(3) Kết luận	(4) Lập luận (ISTQB)	(5) Phần sinh viên sửa
Artifact #3 — Thiết kế luồng E2E + điểm correlation (đã đối chiếu đặc tả) Công cụ: Claude Opus 5 (Claude Code, tiện ích VS Code) Thời điểm: 22:34 12/08/2026 Prompt (nguyên văn): "I have uploada the @apispec.md this contain all the apis you can take a look at it to make sure the api is exist" (sau đó) "Okay use your workflow and write the prompt suggest workflow to ai-audit for hw5"	Dựng lại luồng nghiệp vụ dựa trên <code>api_spec.md</code>, mỗi bước đều kèm mục tham chiếu: 1) <code>POST /api/login</code> §1.2 <i>auth — heavy</i> → 2) <code>GET /api/users/me</code> §2.1 <i>read — heavy</i> → 3) <code>GET /api/orders/my-orders</code> §4.4 <i>read — heavy</i> → 4) <code>PUT /api/users/me</code> §2.2 <i>transactional</i> → 5) <code>POST /api/apply-coupon</code> §5.1 <i>transactional</i> . Bổ sung các điểm correlation (<code>\$.token</code> → header Bearer cho bước 2–5; <code>\$.user.id</code> → body bước 5; <code>#[0].id</code> → bước hủy đơn tùy chọn), assertion cho từng bước, thiết kế 3 file CSV, và ghi toàn bộ vào <code>Main_Report.md</code> §1, §3.2 và §3.3. AI chủ động nêu ba rủi ro chưa giải quyết: khả năng trùng bước 3 và 5 với bạn cùng nhóm, nghi vấn liệu <code>apply-coupon</code> có thật sự ghi xuống CSDL, và việc không có endpoint reset lockout nào được ghi trong tài liệu.	INCOMPLETE	Cả năm endpoint giờ đã đối chiếu khớp với API spec, và việc phân nhóm là hợp lý. Tuy nhiên còn hai khoảng trống mà chỉ tôi mới khép lại được. (a) <code>POST /api/apply-coupon</code> đang được xếp vào nhóm transactional, nhưng §5.1 mô tả nó <i>tính toán discount_amount / final_amount</i> — có thể hoàn toàn không ghi dữ liệu — trong khi §6.4 lại định nghĩa <code>max_uses_per_user</code>, hàm ý có lưu vết ở đâu đó. Chứng nào chưa quan sát thực tế, nhóm transactional có nguy cơ chỉ được bao phủ bởi một endpoint duy nhất. (b) Ảnh chụp của bạn cùng nhóm có <code>/api/orders/my-orders</code> và <code>/api/apply-coupon</code>, nên yêu cầu không trùng lặp vẫn chưa được xác nhận. ISTQB FL §1.4.4 — các giả định trong thiết kế kiểm thử phải được đối chiếu với đối tượng kiểm thử thực tế; §5.2.3 — rủi ro về test basis và phụ thuộc phải được đưa vào số rủi ro, không được để ngấm trong thiết kế.	<Ghi lại phần bạn thực sự đã làm: (1) kiểm tra handler coupon / CSDL để kết luận bước 5 có ghi dữ liệu hay không, nếu không thì thay bằng <code>PUT /api/orders/:id/cancel</code> §4.6; (2) xác nhận quyền sở hữu luồng với hai bạn cùng nhóm và ghi kết quả vào <code>Main_Report.md</code> §1; (3) thêm <code>If Controller</code> với điều kiện <code>{orderId} != NOT_FOUND</code> để các tài khoản mới seed chưa có đơn hàng không làm tăng tỉ lệ lỗi; (4) đối chiếu số dòng <code>users.csv</code> ≥ số VU đình trước mỗi lần chạy.>
Artifact #4 — Dữ liệu kiểm thử CSV (<code>users.csv</code>, <code>profiles.csv</code>, <code>coupons.csv</code>) Công cụ: Claude Opus 5 (Claude Code, tiện ích VS Code) Thời điểm: 10:50 13/08/2026 Prompt (nguyên văn): "Now write the task 1 test plan for loading and let me check" (ba file CSV được sinh ra trong cùng lượt trả lời với file <code>JMX Load</code>)	Sinh ba file trong <code>hw5/data/</code>: <code>users.csv</code> gồm 60 dòng <code>email,password</code> (<code>perf01@test.com ... perf60@test.com</code>, mật khẩu <code>Password123!</code>); <code>profiles.csv</code> gồm 30 dòng <code>name,shipping_address,phone</code> với giá trị khác nhau từng dòng; <code>coupons.csv</code> gồm 5 dòng <code>code,total_amount</code> dùng hai mã <code>SAVE10</code> và <code>TET2025</code>. Số dòng <code>users.csv</code> được chọn là 60 để lớn hơn 50 VU đỉnh của kịch bản Load, khớp với lập luận về <code>recycle=false</code> đã ghi ở <code>Main_Report.md</code> §3.3.	INCOMPLETE	Cấu trúc file và cách phân bổ số dòng đúng theo thiết kế ở §3.3, và việc <code>users.csv</code> (60 dòng) lớn hơn số VU đỉnh (50) đảm bảo không có hai VU dùng chung tài khoản — đúng yêu cầu tránh khóa dây chuyền theo FR-02. Tuy nhiên toàn bộ 60 tài khoản này chưa hề tồn tại trong cơ sở dữ liệu của SUT: đây mới chỉ là dữ liệu trên giấy. Nếu chạy ngay, cả 5 bước của mọi VU đều thất bại từ bước 1. Tương tự, hai mã giảm giá <code>SAVE10</code> và <code>TET2025</code> được lấy từ ví dụ trong API spec §5.1 và §6.4 chứ chưa xác nhận là có thật và còn hiệu lực. ISTQB FL §4.2.3 — kiểm thử data-driven đòi hỏi tập dữ liệu phải phủ hết số lần thực thi dự kiến; §5.1 —	<Ghi lại phần bạn thực sự đã làm: (1) seed đủ 60 tài khoản vào CSDL của SUT — ghi rõ script hoặc câu lệnh SQL đã dùng; (2) kiểm tra hai mã <code>SAVE10</code> / <code>TET2025</code> có tồn tại, chưa hết hạn và <code>max_uses_per_user</code> đủ lớn cho số vòng lặp của bài test, nếu không thì thay bằng mã thật đã tạo; (3) đối chiếu lại số dòng <code>users.csv</code> mỗi khi thay đổi số VU đình ở §3.4.>

(1) Prompt + Công cụ	(2) Output của AI	(3) Kết luận	(4) Lập luận (ISTQB)	(5) Phần sinh viên sửa
Artifact #5 — Test plan Load 23127344_Load_20260812.jmx Công cụ: Claude Opus 5 (Claude Code, tiện ích VS Code) Thời điểm: 10:50 13/08/2026 Prompt (nguyên văn): "Now write the task 1 test plan for loading and let me check"	Sinh file JMX hoàn chỉnh cho JMeter 5.6.3 gồm: Thread Group 50 VU / ramp-up 60s / duration 600s (mọi tham số đều override được qua <code>-Jvusers</code>, <code>-Jrampup</code>, <code>-Jduration</code>); ba CSV Data Set đúng chế độ chia sẻ đã thiết kế; năm HTTP Sampler theo luồng đã chốt ở Artifact #3; Uniform Random Timer 1,5–4 giây cho từng bước; JSON Extractor lấy <code>authToken</code>, <code>userId</code>, <code>orderId</code>; chín assertion trong đó có assertion so khớp <code>\$.email</code> với <code>\${email}</code> từ CSV; một If Controller bọc bước 2–5 với điều kiện <code>"\${authToken}" != "TOKEN_NOT_FOUND"</code>; và listener Summary Report (loại view dành riêng cho kịch bản Load). AI cũng tự kiểm tra file bằng script: XML hợp lệ, các phần tử <code>hashTree</code> ghép đôi đúng, đếm đủ 5 sampler / 3 CSV / 9 assertion. AI chủ động liệt kê bốn điểm chưa kiểm chứng được.	INCOMPLETE	dữ liệu kiểm thử và môi trường phải sẵn sàng trước khi thực thi, nếu không kết quả đo được sẽ phản ánh tình trạng thiếu dữ liệu chứ không phản ánh hiệu năng.	
			Cấu trúc file đúng chuẩn và các tham số kịch bản đều có lập luận (ramp-up 60s cho 50 VU để đo steady state chứ không đo chi phí thiết lập kết nối; think time ngẫu nhiên thay vì cố định). Điểm cộng đáng ghi nhận: If Controller ngăn 4 request rác mỗi vòng lặp khi login hỏng, tránh làm sai lệch tỉ lệ lỗi. Tuy vậy file này chưa từng được mở hay chạy bằng JMeter thật — máy chưa cài JMeter nên việc kiểm tra chỉ dừng ở mức phân tích XML, không thể phát hiện lỗi tương thích schema hoặc phần tử bị JMeter từ chối. Nghiêm trọng hơn, hai JSON Path <code>\$.token</code> và <code>\$.user.id</code> là suy đoán từ mô tả văn xuôi của API spec §1.2 ("trả về chuỗi JWT token và thông tin user") chứ chưa đối chiếu response thật; nếu cấu trúc JSON khác đi thì <code>authToken</code> sẽ luôn bằng <code>TOKEN_NOT_FOUND</code>, If Controller sẽ chặn toàn bộ bước 2–5, và bài test tuy "chạy xong" nhưng chỉ đo mỗi endpoint login. Đây đúng là kiểu lỗi mà ISTQB FL §1.4.4 cảnh báo: giả định trong thiết kế phải được đối chiếu với đối tượng kiểm thử thực tế. §5.1 cũng áp dụng: chưa có môi trường thì chưa thể coi test plan là hoàn chỉnh.	<Ghi lại phần bạn thực sự đã làm: (1) khởi động SUT, đăng nhập thủ công một lần bằng curl/Postman và đối chiếu cấu trúc JSON thật với hai JSON Path <code>\$.token</code> / <code>\$.user.id</code>, sửa lại nếu khác; (2) cài JMeter, mở file để xác nhận không có phần tử nào bị từ chối; (3) chạy thử 1 VU / 1 vòng lặp và kiểm tra cả 5 bước đều pass trước khi chạy đủ tải; (4) chốt kết luận cho bước 5 <code>apply-coupon</code> (vấn đề tồn đọng §1) và sửa lại nhóm phân loại nếu cần; (5) đối chiếu con số 50 VU với năng lực phần cứng thật sau lần chạy đầu tiên, điều chỉnh nếu load generator trở thành nút thắt.>
Artifact #6 — Test plan Stress 23127344_Stress_20260812.jmx Công cụ: Claude Opus 5 (Claude Code, tiện ích VS Code) Thời điểm: 13:44 13/08/2026 Prompt (nguyên văn): "Now implement testplan for stress"	Sinh file JMX kịch bản Stress dùng lại nguyên thân workflow 5 bước của Artifact #5, chỉ thay hồ sơ tải và listener. Hồ sơ tăng tải theo bậc: 5 Thread Group riêng, mỗi group +20 VU với <code>delay</code> lệch nhau 120 giây (0 / 120 / 240 / 360 / 480), thời lượng thu dần (600 / 480 / 360 / 240 / 120 giây) để mọi bậc cùng kết thúc tại giây 600 — cộng dồn từ 20 lên 100 VU. Listener đổi sang Aggregate Report (loại view thứ hai, khác Summary Report của Load). AI chọn cách dùng Thread Group	INCOMPLETE	Hồ sơ tải theo bậc là hợp lý cho mục tiêu tìm điểm knee, và mỗi bậc giữ 120 giây là đủ để hệ thống ổn định trước khi tăng tiếp. Việc tránh phụ thuộc plugin cũng là quyết định đúng cho môi trường chấm bài. Tuy nhiên AI đã tạo ra một lỗi nghiêm trọng mà chính nó phát hiện	Đã mở rộng <code>users.csv</code> từ 60 lên 120 dòng (<code>perf001 ... perf120</code>) và <code>profiles.csv</code> từ 30 lên 60 dòng, sau đó đối chiếu lại bằng script: <code>120 ≥ 100</code> VU đỉnh của Stress. Ghi rõ nguyên tắc vào <code>Main_Report.md</code> §3.3: số dòng <code>users.csv</code> phải tính theo kịch bản nặng nhất, không phải theo riêng kịch bản Load. <Phần bạn cần làm tiếp: (1) seed đủ 120 tài khoản <code>perf001...perf120</code> vào

(1) Prompt + Công cụ	(2) Output của AI	(3) Kết luận	(4) Lập luận (ISTQB)	(5) Phần sinh viên sửa
	chuẩn thay vì Concurrency Thread Group để không phụ thuộc plugin Custom Thread Groups , giúp file mở được trên bản JMeter gốc. Tự kiểm tra bằng script: XML hợp lệ, hashTree ghép đôi đúng, 5 Thread Group với delay/duration đúng như thiết kế, 25 sampler (5 bậc × 5 bước).		khí tự kiểm tra sau đó: file <code>users.csv</code> lúc đó chỉ có 60 dòng — vừa đủ cho Load 50 VU nhưng thiếu cho Stress 100 VU . Với <code>recycle=false</code> và <code>stopThread=true</code> , bậc 4 và bậc 5 sẽ tự tắt thread ngay khi khởi động vì hết dữ liệu, khiến bài test vẫn "chạy xong" và xuất ra file <code>.jtl</code> bình thường nhưng thực tế chỉ đo được khoảng 60 VU thay vì 100 — một sai số âm thầm, không có thông báo lỗi nào trong report. Đây đúng là rủi ro ISTQB FL \$4.2.3 cảnh báo: tập dữ liệu phải phủ hết số lần thực thi dự kiến. \$5.1 cũng áp dụng: dữ liệu kiểm thử là một phần của môi trường, thiếu nó thì kết quả đo phản ánh tình trạng thiếu dữ liệu chứ không phản ánh hiệu năng. Ngoài ra file này vẫn kế thừa mọi rủi ro chưa kiểm chứng của Artifact #5 (chưa mở bằng JMeter thật, hai JSON Path còn là suy đoán).	<i>CSDL — lưu ý cách đánh số đã đổi từ 2 chữ số sang 3 chữ số; (2) sau lần chạy Stress đầu tiên, kiểm tra cột <code>aLLThreads</code> trong file <code>.jtl</code> có thật sự đạt 100 ở bậc cuối hay không, đây là cách xác nhận trực tiếp rằng lỗi trên đã được khắc phục; (3) nếu phần cứng không tải nổi 100 VU thì giảm - <code>Jstepvusers</code> và ghi lại lý do vào \$2.3.></i>
Artifact #7 — Test plan Spike (bản đầu, sau này bị thay thế ở Artifact #10) Công cụ: Claude Opus 5 (Claude Code, tiện ích VS Code) Thời điểm: 14:04 13/08/2026 Prompt (nguyên văn): "Now implement testplan for spike"	Sinh file JMX kịch bản Spike dùng lại nguyên thân workflow 5 bước, gồm 3 Thread Group nối tiếp: giai đoạn 1 nền trước 10 VU (giây 0–120, ramp 20s), giai đoạn 2 spike 100 VU (giây 120–180, ramp chỉ 5 giây), giai đoạn 3 nền sau 10 VU (giây 180–420, ramp 20s). Listener đổi sang View Results Tree đặt <code>error_logging=true</code> — loại view thứ ba, khác Summary Report của Load và Aggregate Report của Stress. AI giải thích hai lựa chọn thiết kế: (a) giai đoạn 3 dài gấp đôi giai đoạn 1 để đo phục hồi, vì so sánh p95 giai đoạn 3 với giai đoạn 1 chính là thước đo hệ thống có trở về mức nền hay không; (b) chỉ ghi sample lỗi để View Results Tree không ngốn RAM ở mức 100 VU, tránh biến load generator thành nút thắt. Tự kiểm tra bằng script: XML hợp lệ, hashTree ghép đôi đúng, 3 Thread Group với delay/duration không chồng lấn, 15 sampler, và đối chiếu 100 VU đỉnh ≤ 120 dòng <code>users.csv</code> .	INCOMPLETE	Hình dạng spike đúng mục tiêu: tỉ lệ tăng gấp 10 lần trong 5 giây là cú sốc đủ mạnh, và việc có giai đoạn phục hồi khắc phục đúng thiếu sót mà biểu mẫu này từng nêu làm ví dụ ("không có giai đoạn phục hồi sau spike thì test plan không thể cho biết SUT có trở về mức nền hay không" — ISTQB FL \$2.2 : một phép kiểm thử phải tạo ra đúng thông tin mà mục tiêu của nó đòi hỏi). Lần này AI đã tự đổi chiều số dòng CSV, không lặp lại lỗi của Artifact #6. Tuy nhiên vẫn còn hai điểm chưa đóng được. (a) Toàn bộ file kế thừa rủi ro chưa kiểm chứng của Artifact #5: chưa từng mở bằng JMeter thật, và hai JSON Path <code>\$.token / \$.user.id</code> vẫn là suy đoán từ mô tả văn xuôi của API spec §1.2. (b) Nghiêm trọng hơn với	<i><Ghi lại phần bạn thực sự đã làm: (1) sau lần chạy Spike đầu tiên, mở file <code>.jtl</code> kiểm tra cột <code>aLLThreads</code> xem có thật sự đạt 100 trong khoảng giây 120–180 hay không — nếu không đạt thì nối <code>-Jspikevusers</code> xuống hoặc tăng ramp lên 10 giây và ghi rõ lý do vào \$2.3; (2) đối chiếu p95 của giai đoạn 1 và giai đoạn 3 để kết luận hệ thống có phục hồi hay không, điền số liệu vào \$3.7; (3) nếu giai đoạn 3 vẫn chưa về mức nền khi hết 240 giây, kéo dài thêm và ghi nhận đây là một phát hiện về hiệu năng; (4) chốt kết luận cho bước 5 <code>apply-coupon</code> (vấn đề tồn đọng \$1) — nếu phải đổi sang <code>PUT /api/orders/:id/cancel</code> thì cả ba file test plan đều phải sửa.></i>

(1) Prompt + Công cụ	(2) Output của AI	(3) Kết luận	(4) Lập luận (ISTQB)	(5) Phần sinh viên sửa
Artifact #8 — Rà soát chéo ba test plan (AI đóng vai người rà soát độc lập) Công cụ: Claude Opus 5 (Claude Code, tiện ích VS Code) Thời điểm: 14:22 13/08/2026 Prompt (nguyên văn): "Rà soát lại cả ba file .jmx đã sinh. Đóng vai một người rà soát độc lập, tìm những lỗi mà kiểm tra XML không phát hiện được."	Phát hiện 4 lỗi ngữ nghĩa mà ba lượt tự kiểm tra trước đó đều báo "OK": (1) <code>users.csv</code> đặt <code>recycle=false</code> + <code>stopThread=true</code> trong khi <code>LoopController.loops = -1</code> — JMeter đọc CSV theo mỗi vòng lặp chứ không phải mỗi thread, nên file 120 dòng cạn sau 120 vòng lặp tính trên toàn bộ thread rồi mọi thread bị giết; ước tính Load cần ~2 264 vòng lặp, Stress ~4 528, Spike ~3 170. (2) Năm <code>UniformRandomTimer</code> đặt ngang hàng với sampler thay vì lồng trong hashTree — JMeter áp dụng mọi timer cho mọi sampler trong cùng scope, khiến think time bị nhân ~5 lần (13,25 s/bước thay vì 1,5–4 s). (3) <code>JSONPostProcessor</code> của <code>userId</code> đặt <code>defaultValues=0</code> — số 0 là <code>user_id</code> hợp lệ về kiểu nên request lỗi vẫn được gửi và assertion vẫn pass. (4) Kịch bản nhiều giai đoạn tính nhu cầu CSV theo VU đỉnh thay vì cộng dồn qua các giai đoạn. AI nhấn mạnh cả bốn lỗi đều thuộc loại "âm thầm": file XML hợp lệ, test vẫn chạy, vẫn xuất <code>.jtl</code>, không có thông báo lỗi nào.	VALID	riêng kịch bản này: ramp 5 giây cho 100 VU có thể vượt khả năng khởi tạo thread của chính JMeter trên phần cứng đang dùng. Nếu JMeter không kịp tạo đủ thread thì "cú spike" đo được là giới hạn của load generator chứ không phải phản ứng của SUT — đúng rồi ro ISTQB FL \$5.1 về môi trường kiểm thử ảnh hưởng tính hợp lệ của kết quả. Con số 100 VU / 5 giây là AI chọn theo thông lệ, chưa hề đo trên máy thật. Đây là lượt có giá trị cao nhất trong tám bước đầu, và nó chứng minh một nguyên tắc của ISTQB FL \$1.4.3 (kiểm thử tĩnh phát hiện khuyết tật mà kiểm thử động khó thấy): cả bốn lỗi đều không làm test thất bại, chúng chỉ làm test đo sai. Một bài test dừng ở giây 30 thay vì 600 vẫn xuất ra <code>.jtl</code> trông bình thường — người đọc kết quả không có cách nào biết. Lập luận của AI về scope của timer khớp đúng tài liệu JMeter (timer là <i>pre-processor</i> áp dụng cho toàn scope, không phải cho phần tử liền trước). Về <code>defaultValues</code>, nguyên tắc "giá trị mặc định phải gây lỗi quan sát được" tương ứng ISTQB FL \$4.4 về thiết kế test case có khả năng phát hiện lỗi: một oracle trả về giá trị hợp lệ khi thất bại là oracle hỏng. Nhận VALID vì cả bốn phát hiện đều được kiểm chứng lại độc lập bằng <code>validate_jmx.py</code> và đều tái lập được.	Tôi yêu cầu AI viết <code>scripts/validate_jmx.py</code> để tự động hoá việc phát hiện bốn lỗi này thay vì rà tay, rồi chạy trên cả ba file. Kết quả: Load 3 lỗi, Stress 3 lỗi × 5 bậc, Spike sạch (vì đã sinh lại bằng skill). Tôi quyết định không sửa Load/Stress ngay tại thời điểm này mà giữ nguyên để làm bằng chứng đối chiếu cho \$3.6, chỉ sửa sau khi đã đối chiếu mã nguồn SUT (Artifact #11) — vì nếu sửa sớm thì mất tư liệu chứng minh AI lập lại cùng một lỗi qua ba lần sinh khác nhau. Ghi toàn bộ vào \$3.6 dòng 3–6.
Artifact #9 — Agent Skill <code>jmeter-testplan-eshop</code> Công cụ: Claude Opus 5 (Claude Code, tiện ích VS Code) Thời điểm: 14:46 13/08/2026 Prompt (nguyên văn): "Đóng gói toàn bộ quy trình sinh test plan JMeter cho EShop thành một Agent Skill tái sử dụng được, mã hoá sẵn bốn lỗi đã gặp để lần sau không lặp lại."	Sinh bộ skill gồm <code>SKILL.md</code> (quy trình 7 bước, bắt buộc hỏi loại kịch bản trước, cấm đoán tên endpoint), bốn file tham chiếu (<code>api_spec.md</code> làm nguồn chân lý duy nhất cho tên endpoint, <code>jmeter-pitfalls.md</code> ghi bốn lỗi kèm cách tránh, <code>scenario-profiles.md</code> tham số mặc định cho Load/Stress/Spike, <code>workflows.md</code> các luồng E2E và phân nhóm endpoint), một <code>templates/base-workflow.md</code> chứa khung XML đã sửa sẵn bốn lỗi, và	VALID	Skill này là hiện thực hoá đúng tinh thần ISTQB FL \$5.1.2 về tài sản kiểm thử tái sử dụng được (testware): quy trình được ghi thành tài liệu có cấu trúc thay vì tồn tại trong đầu một người. Giá trị lớn nhất không nằm ở phần sinh code mà ở phần mã hoá tri thức thất bại — bốn lỗi	Tôi rà lại toàn bộ nội dung skill để đảm bảo không có endpoint nào bị bỏ. Bổ sung yêu cầu mỗi test plan phải dùng một loại listener khác nhau vào <code>SKILL.md</code> (HW05 mục 6 Task 1 bắt buộc ba report view không lặp lại) — AI ban đầu không nêu ràng buộc này. Kiểm chứng skill bằng cách dùng nó sinh lại kịch bản Spike (Artifact #10) và đối

(1) Prompt + Công cụ	(2) Output của AI	(3) Kết luận	(4) Lập luận (ISTQB)	(5) Phần sinh viên sửa
Artifact #10 — Sinh lại test plan Spike bằng skill (23127344_Spike_20260813.jmx) Công cụ: Claude Opus 5 (Claude Code, tiện ích VS Code) Thời điểm: 23:19 13/08/2026 Prompt (nguyên văn): "Use @hw5/skills/jmeter-testplan-eshop/SKILL.md to generate the jmx testplan for spike technique"	<code>scripts/validate_jmx.py</code> . Điểm đáng chú ý: skill chủ động đánh dấu 🚩 ba giả định chưa kiểm chứng — cấu trúc JSON của response login, việc apply-coupon có ghi CSDL hay không, và cơ chế reset bộ đếm lockout — kèm chỉ dẫn "dừng lại và hỏi người dùng" thay vì tự quyết.		Ở Artifact #8 được viết vào <code>jmeter-pitfalls.md</code> kèm triệu chứng, nguyên nhân và cách sửa, nên lần sinh sau không lặp lại. Việc skill tự đánh dấu giả định chưa kiểm chứng thay vì im lặng đoán bừa là điểm mạnh nhất: nó biến một rủi ro ngầm thành một câu hỏi hiện rõ, đúng nguyên tắc ISTQB FL \$1.2 về việc kiểm thử phải dựa trên test basis xác định chứ không dựa vào giả định.	chiếu xem có tránh được bốn lỗi cũ không.
	Skill hoạt động đúng quy trình: đọc bốn file tham chiếu, bỏ qua câu hỏi loại kịch bản (vì prompt đã nêu rõ "spike"), tính tổng nhu cầu CSV cộng dồn qua ba giai đoạn (~554 vòng lặp trên 120 dòng → bắt buộc recycle=true), rồi dừng lại hỏi bốn câu trước khi sinh file: load generator có chạy cùng máy với SUT không, đã xác minh cấu trúc JSON của login chưa, đã đọc mã nguồn xác nhận cơ chế lockout chưa, và apply-coupon có ghi CSDL không. Sau khi nhận trả lời, sinh file với SPIKE_VUSERS mặc định 60 thay vì 100 (vì load generator chạy chung máy), timer lồng đúng trong hashTree, defaultValues đều là chuỗi *_NOT_FOUND , và ghi bốn điều kiện chưa kiểm chứng vào TestPlan.comments . File sạch cả bốn lỗi cũ ngay từ lần sinh đầu.	VALID (cho phần tránh lỗi cũ) — nhưng lượt này sinh ra một lỗi mới được ghi riêng ở \$3.6 dòng 10.	Skill chứng minh được giá trị: bốn lỗi ở Artifact #8 không tái xuất hiện, và việc giảm VU đỉnh từ 100 xuống 60 là quyết định dựa trên bối cảnh thực tế (JMeter cạnh tranh CPU với SUT) chứ không theo mặc định — đúng ISTQB FL \$5.1 về ảnh hưởng của môi trường kiểm thử tới tính hợp lệ của kết quả. Việc skill hỏi bốn câu trước khi sinh thay vì đoán là hành vi đúng theo ISTQB FL \$1.2 . Tuy nhiên lượt này bộc lộ một hiện tượng đáng ghi nhận: khi áp dụng nguyên tắc "assertion ghi dữ liệu phải kiểm server trả lại đúng giá trị vừa ghi" (đúng theo <code>workflows.md</code>), AI không kiểm chứng endpoint cụ thể có hành xử như vậy không — PUT /api/users/me thực tế chỉ trả <code>{"message": "Profile updated"}</code> . Một nguyên tắc đúng, áp dụng mù, vẫn tạo ra lỗi. Đây là minh họa cho giới hạn của việc mã hoá tri thức thành quy tắc: quy tắc không thay thế được việc quan sát hệ thống thật.	Tôi chấp nhận file nhưng ghi nhận lỗi mới vào \$3.6 dòng 10. Đáng chú ý là lỗi này chỉ lộ ra ở lượt đối chiếu mã nguồn (Artifact #11) chứ không phải khi rà file — vì cả JSON Path lẫn giá trị mong đợi đều đúng cú pháp. Tôi cũng cho commit file này thay thế bản 20260812 (git ghi nhận là rename + sửa, 60% tương đồng) để giữ lịch sử liên mạch.
Artifact #11 — Đối chiếu ba test plan với mã nguồn SUT Công cụ: Claude Opus 5 (Claude Code, tiện ích VS Code) Thời điểm: 23:42 13/08/2026 Prompt (nguyên văn): "Bây giờ bạn hãy review lại 3 file plan jmeter cho tôi luôn được"	Đọc <code>backend/server.js</code> và <code>backend/database.js</code> , truy vấn trực tiếp <code>database.sqlite</code> , phát hiện 4 lỗi mới mà mọi lần kiểm tra <code>.jmx</code> trước đó đều bỏ sót: (1) <code>users.csv</code> dùng 120 tài khoản perf* không tồn tại trong CSDL — <code>database.js:91-94</code> chỉ seed hai tài khoản, truy vấn xác nhận COUNT(*)	VALID	Đây là bước quyết định của toàn bài. Hai lỗi đầu không nằm trong file .jmx nên <code>validate_jmx.py</code> về nguyên lý không thể phát hiện — script chỉ đọc XML, nó không biết	Tôi yêu cầu AI viết <code>data/seed_perf_users.py</code> (idempotent, có cờ <code>--reset</code> để mở khoá giữa các lần chạy Stress/Spike theo yêu cầu HW05 mục 6 Task 1), chạy và xác minh 120/120 tài khoản. Thay toàn bộ <code>coupons.csv</code> bằng 6 dòng đối

(1) Prompt + Công cụ	(2) Output của AI	(3) Kết luận	(4) Lập luận (ISTQB)	(5) Phần sinh viên sửa
không. tôi đã đính kèm src code ở folder @group05_eshop để bạn tham khảo và viết file Jmeter cho đúng hơn. và nếu có lỗi thì ghi đó @hw5/Main_Report.md nếu cần thiết"	WHERE email LIKE 'perf%' trả về 0; (2) coupons.csv có 4/5 dòng sẽ fail — mã TET2025 chỉ là ví dụ minh họa trong api_spec.md \$6.4 chứ không có trong seed (404), và SAVE10 với total_amount=250000 dưới min_order_amount=300000 (400); (3) assertion PUT /api/users/me kiểm body chứa số điện thoại vừa ghi sẽ fail 100% vì server.js:131-134 chỉ trả {"message": "Profile updated"}; (4) POST /api/apply-coupon bị gán nhãn [transactional] sai — server.js:363-441 chỉ SELECT rồi tính, bảng coupon_usage chỉ được ghi bởi endpoint khác. Đồng thời xác minh dứt điểm ba giả định treo từ Artifact #3: \$.token, \$.user.id, \$.email đều đúng; và server.js:47-50 reset bộ đếm lockout khi đăng nhập thành công nên recycle=true an toàn.		gi về nội dung CSDL. Điều này minh họa chính xác ISTQB FL \$1.4.2 về việc kiểm thử phải dựa trên test basis đầy đủ: api_spec.md mô tả schema của API nhưng không mô tả dữ liệu đã seed, nên một test plan đúng hoàn toàn so với đặc tả vẫn có thể vô hiệu hoàn toàn khi chạy thật. Lỗi (2) là dạng nhầm lẫn đặc trưng khi tài liệu vừa mô tả schema vừa chứa sample values — AI coi ví dụ minh họa là dữ liệu có thật. Lỗi (4) từng được skill đánh dấu ⚠️ "chưa xác minh" ngay từ đầu vì api_spec.md \$5.1 và \$6.4 mâu thuẫn nhau; skill đã đúng khi không tự quyết, và chỉ mã nguồn mới trả lời dứt khoát được. Nếu bỏ qua lượt này, cả ba bài test vẫn chạy và vẫn xuất .jtl đầy đủ nhưng số liệu hoàn toàn vô nghĩa.	chiều trực tiếp với database.js:107-110, kiểm chứng từng dòng bằng script mô phỏng đúng logic server.js:379 (lưu ý toán tử > chứ không phải >=). Quyết định thêm bước 4b GET /api/users/me để xác minh lệnh UPDATE đã commit, thay vì bỏ assertion. Áp cả ba bản sửa của Spike vào Load và Stress (chuyển 30 timer vào hashTree). Ghi 4 lỗi vào \$3.6 dòng 8–11.

Artifact #12 — Cài JMeter, xác minh luồng và phát hiện lỗi assertion \$.phone Công cụ: Claude Opus 5 (Claude Code, tiện ích VS Code) Thời điểm: 00:12–00:26 14/08/2026 Prompt (nguyên văn): "Bạn tự cài JMeter cho tôi luôn được không" → sau đó "Bây giờ bạn hãy từ verify lại 3 file plan jmeter cho tôi luôn được không. tôi đã chạy sẵn fe và be rồi"	Tải Apache JMeter 5.6.3 từ CDN chính thức, đối chiếu SHA-512 khớp với checksum công bố trước khi giải nén. Xác nhận cả ba .jmx mở được bằng JMeter thật (Load và Spike đạt Starting standalone test, Stress đạt Created the tree successfully). Viết scripts/verify_flow.py kiểm chứng từng bước bằng request trực tiếp tới SUT: 13/13 assertion PASS. Nhưng smoke test 1 VU qua JMeter lại fail 17,65%, toàn bộ ở bước 4b, với thông báo: Value in json path '\$.phone' expected to be '0912345001', but found '0912345001' — hai giá trị hiển thị giống hệt nhau nhưng vẫn fail. Nguyên nhân: JSONVALIDATION=true + ISREGEX=false khiến JMeter so sánh theo kiểu dữ liệu, \$.phone là chuỗi còn giá trị mong đợi bị đọc như số nên mất số 0 đứng đầu. Sửa bằng ISREGEX=true ở cả ba file (9 assertion) → smoke đạt đủ 6 nhãn, 0% lỗi.	VALID (cho phần cài đặt và phát hiện lỗi) — lỗi \$.phone được ghi riêng ở \$3.6 dòng 12.	Lượt này bổ sung tăng kiểm chứng thử năm cho bảng ở \$3.1: chạy thật trên SUT thật. Nó chứng minh một điều mà bốn tăng trước không thể — validate_jmx.py chỉ đọc XML nên không biết JMeter có chấp nhận từng guiclass/testclass hay không, và cũng không biết assertion sẽ hành xử ra sao khi gặp dữ liệu thật. Lỗi \$.phone là ví dụ điển hình của khuyết tật chỉ lộ ra khi thực thi (ISTQB FL \$1.4.3): cả JSON Path lẫn giá trị mong đợi đều đúng cú pháp, không công cụ tính nào bắt được. Nguyên nhân sâu xa đáng ghi nhận: số điện thoại Việt Nam có số 0 đứng đầu — một đặc điểm của dữ liệu bản địa mà mặc định của công cụ quốc tế không lường trước, tương ứng rủi ro về equivalence partitioning ở ISTQB FL \$4.2 khi dữ liệu thật không nằm trong phân vùng mà công cụ giả	Tôi yêu cầu chạy smoke 1 VU trước khi chạy tải thật — chính bước này bắt được lỗi. Nếu bỏ qua và chạy thẳng ba kịch bản, cả ba file .jtl sẽ có tỉ lệ lỗi ~17% do một assertion hỏng, và toàn bộ phân tích ở Task 2 sẽ dựa trên số liệu sai. Tôi đưa verify_flow.py vào repo làm bằng chứng tái lập được, và bổ sung bước này thành công chặn bắt buộc trong RUNBOOK.md trước mỗi lần chạy tải.
---	---	--	--	--

(1) Prompt + Công cụ	(2) Output của AI	(3) Kết luận	(4) Lập luận (ISTQB)	(5) Phần sinh viên sửa
Artifact #13 — Chạy bốn kịch bản trên SUT thật và sự cố CSDL bị xóa Công cụ: Claude Opus 5 (Claude Code, tiện ích VS Code) + Apache JMeter 5.6.3 Thời điểm: 06:39–07:38 14/08/2026 Prompt (nguyên văn): "Giả sử bạn là tôi. tự chạy luôn jmeter và đưa ra kết quả rồi hoàn thiện task1 cho tôi., còn video thì tôi sẽ quay sao tại giờ không phù hợp"	Lần chạy Load đầu tiên thất bại: 100% lỗi trên 14 229 sample, chỉ 1 nhấn duy nhất. Truy nguyên: <code>database.js:117</code> gọi <code>initDatabase()</code> ở top level, hàm này <code>DROP TABLE</code> cả 6 bảng rồi seed lại 2 tài khoản mặc định; <code>server.js:4</code> require file đó nên mỗi lần backend khởi động lại là 120 tài khoản perf* biến mất. Đối chiếu thời gian xác nhận: tiến trình <code>node.exe</code> khởi động lúc 06:39:29, <code>mtime</code> của <code>database.sqlite</code> cũng đúng 06:39:29. Sau khi seed lại và thêm bước <code>verify_flow.py</code> làm cổng chặn, chạy lại thành công cả bốn kịch bản: Load 11 011 sample / 18,4 req/s, Stress 13 329 / 22,3 req/s (đạt đủ 100 VU ở cả 5 bậc), Spike 2 620 / 6,3 req/s (đạt đủ 60 VU, tỉ lệ phục hồi p95(GD3)/p95(GD1) = 1,00), Endurance 567 174 sample / 630,3 req/s trong 15 phút. Tất cả 0% lỗi. RSS backend đi ngang 102–104 MB rồi tụt về 59,7 MB khi tải dừng → không rò rỉ.	VALID	định. Đây cũng là lỗi thứ hai liên tiếp do AI tạo ra trong lúc đang sửa lỗi khác (#10 → thêm bước 4b → #12). Sự cố CSDL là minh chứng thực tế cho lỗi #8 ở §3.6, và nó xảy ra sau khi lỗi đó đã được ghi nhận — nghĩa là biết trước rủi ro vẫn không đủ, phải có cơ chế kiểm tra tự động. Điều này cũng cố nguyên tắc ISTQB FL \$5.1 về kiểm soát môi trường kiểm thử: một môi trường thay đổi giữa chừng làm vô hiệu toàn bộ phép đo mà không phát ra tín hiệu nào. Việc <code>check_jt1.py</code> bắt được ngay (cảnh báo "chỉ thấy 1 nhấn, mong đợi 6") cho thấy giá trị của việc kiểm tra tính hợp lệ của phép đo trước khi diễn giải kết quả. Về phần số liệu: Stress không tìm được điểm gãy trong dải 20–100 VU (p95 giữ nguyên 2–3 ms) — đây là kết quả âm tính hợp lệ, không phải test thất bại, và việc ghi nhận trung thực điều đó quan trọng hơn việc cố ép ra một kết luận.	Tôi yêu cầu AI chạy lại từ đầu sau sự cố, và bổ sung <code>verify_flow.py</code> thành bước bắt buộc trước mỗi lần chạy. Tự phân tích số liệu và phát hiện: bước login chỉ mất 2,2 ms trong khi §3.2 lập luận auth-heavy là nhóm nặng CPU — kiểm tra <code>server.js:46</code> xác nhận SUT so sánh mật khẩu bằng <code>===</code> trên chuỗi plaintext, <code>package.json</code> không có <code>bcrypt/argon/scrypt</code> nào. Ghi nhận giả định "auth-heavy = nặng CPU" không đúng với SUT này vào §3.7. Quyết định tạo thêm biến thể Endurance với think time 50–100 ms để tìm trần throughput thật thay vì trần VU. Nén <code>.jt1</code> endurance 82 MB → 4,2 MB <code>.gz</code> để commit được lên GitHub (giải nén ra đủ 567 175 dòng).
Artifact #14 — Phân tích .jt1 và đề xuất ngưỡng hiệu năng (Task 2) Công cụ: Claude Opus 5 (Claude Code, tiện ích VS Code) Thời điểm: 09:17 14/08/2026 Prompt (nguyên văn): "Đây là kết quả 4 kịch bản kiểm thử hiệu năng trên một backend REST API. Hãy phân tích và đề xuất các ngưỡng hiệu năng (SLO) phù hợp, kèm khuyến nghị tối ưu." (AI được đưa bảng thống kê tổng hợp từ 4 file <code>.jt1</code>, KHÔNG được đưa mã nguồn SUT và KHÔNG được đưa nội dung §3.6)	Đánh giá "hiệu năng xuất sắc trên cả bốn kịch bản". Bốn khẳng định chính: (a) "chịu được 100 người dùng đồng thời mà không suy giảm → kiến trúc mở rộng tốt"; (b) "630 req/s xác nhận đây là throughput tối đa hệ thống có thể xử lý ổn định"; (c) "max 177 ms nhiều khả năng là dấu hiệu truy vấn CSDL chậm không có index"; (d) "throughput 18–22 req/s ở ba kịch bản đầu tương đối thấp, cho thấy có thể tồn tại nút thắt ở tầng kết nối". Đề xuất SLO: p95 < 10 ms, p99 < 20 ms, lỗi < 0,1%, throughput ≥ 600 req/s, số VU đồng thời ≥ 100. Khi được hỏi tiếp về tối ưu, đưa 5 khuyến nghị: thêm index, bật SQLite WAL, tăng connection pool, thêm cache Redis, chuyển sang microservice.	INVALID (6 diễn giải sai) — chỉ tiết ở §4.3	Điểm đáng chú ý nhất: mọi con số AI trích dẫn đều đúng, sáu điểm sai đều là lỗi diễn giải chứ không phải lỗi số học. Ba nhóm: (1) <i>Nhầm giá trị quan sát được với giới hạn</i> — "cao nhất đo được" bị đọc thành "tối đa có thể". Tại 630 req/s, p95 vẫn 3 ms, lỗi 0%, CPU mới dùng ~52% của một nhân; không chỉ số nào chạm giới hạn. (2) <i>Suy diễn nguyên nhân mà không kiểm tra phân bố</i> — 13 sample vượt 50 ms tụm vào đúng hai thời điểm (giây 244 và 459) với nhiều nhấn khác nhau cùng chậm một lúc, đặc trưng của GC pause chứ không phải truy vấn thiếu index (nếu thiếu index thì các sample chậm phải tập trung vào một nhấn và rải đều theo thời gian).	Khai báo minh bạch: theo lựa chọn của tôi, phần phân tích (§4.2) và phần phản biện (§4.3) do cùng một AI viết. Tôi ghi rõ hạn chế này ngay đầu §4 thay vì che giấu, và yêu cầu §4.2 được sinh dưới ràng buộc chỉ nhìn bảng số liệu, không dùng hiểu biết về mã nguồn hay §3.6. Tôi tự kiểm chứng lại từng con số phản biện bằng cách chạy <code>check_jt1.py</code> và mở <code>.jt1</code> lọc <code>elapsed > 50</code> đối chiếu cột <code>timeStamp</code> — chính bước này xác nhận giả thuyết GC pause. Đặt lại ngưỡng ở §4.4 theo từng chế độ tải thay vì một ngưỡng chung, và cố ý không đặt SLO theo số VU kèm giải thích lý do.

(1) Prompt + Công cụ	(2) Output của AI	(3) Kết luận	(4) Lập luận (ISTQB)	(5) Phần sinh viên sửa
			(3) <i>Nhằm thước đo với thứ được đo</i> — số VU bị coi là mức tải, trong khi think time chiếm 99,89% thời gian mỗi vòng lặp nên 100 VU ở Stress chỉ tạo 22,3 req/s, còn 50 VU ở Endurance cho 630,3 req/s (gấp 28,3 lần). Theo ISTQB FL \$5.3, một chỉ số chỉ có ý nghĩa khi hiểu rõ điều kiện sinh ra nó; đặt SLO theo số VU sẽ tạo ra ngưỡng có thể đạt được bằng cách tăng think time — một chỉ số tự lừa dối. Về khuyến nghị tối ưu: 1 khả thi có điều kiện / 4 ảo giác — index vô ích trên bảng 122 dòng nằm gọn trong page cache; sqlite3 dùng một đối tượng db duy nhất (<code>database.js:5</code>) nên không có connection pool nào để tăng; Redis không dùng được vì dữ liệu riêng theo user và p95 hiện đã là 2 ms; microservice không có căn cứ khi CPU mới dùng nửa nhân trên 12 luồng.	
Artifact #15 — Đề xuất Continuous Performance Testing (Task 3) Công cụ: Claude Opus 5 (Claude Code, tiện ích VS Code) Thời điểm: 09:23 14/08/2026 Prompt (nguyên văn): "Làm luôn task 3 cho tôi"	Sinh mô hình CPT ba tầng (smoke 2 phút cho PR / load 10 phút cho <code>main</code> / soak 15 phút hằng đêm), lưu đồ mermaid có nhánh reset CSDL và cổng <code>verify_flow.py</code>, bảng chín đánh đổi. Phát hiện quan trọng nhất: quy tắc <code>p95 > baseline × 1,2</code> trong khung báo cáo ban đầu không dùng được với SUT này. Bảng chứng: p95 tính theo từng cửa sổ 60 giây trong cùng một lần chạy Endurance cho <code>[2,3,3,3,3,3,3,3,2,3,3,3,3,3,3]</code> ms — dao động ±50% hoàn toàn do JMeter ghi <code>elapsed</code> theo mili-giây nguyên, không đo bất kỳ thay đổi nào. Với baseline 2 ms, ngưỡng $×1,2 = 2,4$ ms nên mọi cửa sổ cho 3 ms đều bị báo hồi quy. Đề xuất thay bằng ngưỡng lai <code>max(baseline + 10 ms, baseline × 1,5)</code>.	VALID	Đây là lượt mà AI tự bác bỏ một quy tắc trong chính khung mà nó đã sinh ra trước đó (Artifact #1), dựa trên số liệu thật thu được sau này. Lập luận về sai số lượng tử hóa là đúng về mặt đo lường: ngưỡng tương đối chỉ có ý nghĩa khi giá trị đủ lớn so với độ phân giải của phép đo — ở đây độ phân giải (1 ms) chiếm tới một phần ba giá trị đo (3 ms). Ngưỡng lai tự thích ứng: khi SUT còn nhanh thì về tuyệt đối chống nhiễu, khi SUT chậm đi thì về tỉ lệ tiếp quản. Việc mô hình phân biệt rõ lỗi môi trường với hồi quy hiệu năng (cổng <code>verify_flow.py</code> dừng pipeline thay vì báo hồi quy) là bài học rút trực tiếp từ Artifact #13, đúng nguyên tắc ISTQB FL \$5.1 về việc kết quả kiểm thử chỉ hợp lệ khi môi trường được kiểm soát.	Tôi kiểm chứng lại toàn bộ phép tính trong \$5.2.1 bằng script: bảng ngưỡng lai đúng cả ba mức baseline (3 ms → tuyệt đối thẳng, 20 ms → hai vế bằng nhau, 100 ms → tỉ lệ thẳng), cáo buộc báo động giả đúng (3 > 2,4), trôi baseline $1,05^{20} = 2,65$ lần. Bổ sung \$5.5 nêu ba điều mô hình này KHÔNG làm được, trong đó quan trọng nhất: pipeline xanh không có nghĩa hệ thống đúng — toàn bộ 594 134 sample đều "thành công" trong khi <code>apply-coupon</code> trả kết quả sai gấp hơn 11 lần (\$3.11).

(1) Prompt + Công cụ	(2) Output của AI	(3) Kết luận	(4) Lập luận (ISTQB)	(5) Phần sinh viên sửa
Artifact #16 — Hướng dẫn quay video và chụp ảnh evidence (EVIDENCE_GUIDE.md) Công cụ: Claude Opus 5 (Claude Code, tiện ích VS Code) Thời điểm: 07:47 14/08/2026 Prompt (nguyên văn): "Bây giờ bạn hãy viết cho tôi 1 guideline hướng dẫn quay video và ảnh chụp làm evidence từng bước 1. tôi phải làm từng cái gì cái gì ở root folder"	Sinh EVIDENCE_GUIDE.md gồm 6 phần: chuẩn bị môi trường, ảnh phần cứng, quay từng kịch bản, kết video, dọn dẹp và upload, tạo GitHub Issue. AI chủ động nêu một cạm bẫy mà tôi chưa nghĩ tới: bốn kịch bản đã chạy xong và số liệu đã trích dẫn trong §3.7/§3.9, nên khi chạy lại để quay video phải ghi ra file demo_*.jtl thay vì ghi đè — nếu ghi đè, con số trong báo cáo sẽ không khớp file .jtl, đúng kiểu sai lệch mà mục 11 của đề bài soi. Kèm kịch bản thuyết minh tiếng Việt cho từng kịch bản với số liệu thật, danh sách 10 file ảnh cần chụp kèm đường dẫn cụ thể, và nội dung GitHub Issue soạn sẵn. Tự kiểm chứng hai lệnh trong hướng dẫn (thêm PATH JMeter, curl kiểm tra SUT) chạy đúng trên máy này trước khi giao.	VALID	Giá trị của lượt này không nằm ở phần soạn văn bản mà ở việc nhận ra một rủi ro về tính toàn vẹn của bằng chứng. Ghi đè .jtl là hành động tự nhiên khi chạy lại, nhưng nó sẽ phá vỡ khả năng truy nguyên giữa báo cáo và dữ liệu thô — đúng thứ mà ISTQB FL §5.3 gọi là traceability giữa kết quả kiểm thử và tài liệu báo cáo. Việc AI tự kiểm chứng lệnh trước khi đưa vào hướng dẫn cũng đáng ghi nhận: một hướng dẫn có lệnh sai còn tệ hơn không có hướng dẫn.	Tôi làm theo hướng dẫn để quay video và chụp ảnh. Trong quá trình đó phát hiện hướng dẫn ban đầu ghi endurance là "tùy chọn, nếu cần thêm thời lượng video" — nghe như phương án chữa cháy. Tôi yêu cầu AI đối chiếu lại đề bài và sửa: dòng 133 (ảnh chụp) chỉ áp dụng cho "cả ba kịch bản" Load/Stress/Spike, còn endurance ở dòng 135 là gạch đầu dòng riêng chỉ yêu cầu "báo cáo kèm số liệu cụ thể". Bổ sung bằng "cần làm gì cho kịch bản nào" để không phải đọc hết mới biết.
Artifact #17 — Bố cục lại Main_Report.md Công cụ: Claude Opus 5 (Claude Code, tiện ích VS Code) Thời điểm: 10:07 14/08/2026 Prompt (nguyên văn): "refactor lại file @hw5/Main_Report.md bố cục lại cho dễ nhìn hơn"	Chuyển §3.6 từ bảng 5 cột (ô dài 1 200–1 500 ký tự, phải cuộn ngang mới đọc được) sang danh sách có cấu trúc: mỗi lỗi thành một mục #### với dòng tóm tắt mức ảnh hưởng + cách phát hiện, rồi bốn đoạn có nhãn đậm. Thêm bảng tra nhanh ở đầu. Dòng dài nhất giảm 1 530 → 800 ký tự. Thêm phần "Tóm tắt kết quả" ngay đầu báo cáo. Viết lại mục lục thành bảng có cột "điểm chính", bổ sung mục con của §4 và §5 trong thẻ <details>. Tách một đoạn văn 1 240 ký tự ở §3.2 thành bảng theo nhóm endpoint. Đồng thời phát hiện ba lỗi thật trong bản gốc: (a) bảng §3.6 đánh số trùng lặp — có hai dòng số 3 và hai dòng số 4; (b) ba dòng vẫn là mẫu chưa điền (<ví dụ: think time = 0...>) lẫn giữa các dòng thật; (c) dòng lỗi 12 bị tách khỏi bảng bởi một dòng trống nên không render cùng bảng.	VALID	Ba lỗi phát hiện được đều thuộc loại chỉ lộ ra khi xử lý dữ liệu một cách có hệ thống, không thấy được khi đọc lướt. Số thứ tự trùng lặp và dòng mẫu còn sót là những khiếm khuyết nhỏ về hình thức nhưng ảnh hưởng trực tiếp tới độ tin cậy của báo cáo — người chấm gặp một dòng <ví dụ: ...> giữa bảng lỗi sẽ nghi ngờ toàn bộ phần còn lại có được rà soát hay không. Việc AI đánh số lại liên tục 1–11 và cập nhật toàn bộ dẫn chiếu chéo (ánh xạ 8→7, 9→8, 10→9, 11→10, 12→11) là thao tác đúng: một tài liệu có dẫn chiếu sai còn khó dùng hơn tài liệu không có dẫn chiếu. Sau khi sửa, AI tự kiểm chứng bằng cách so sánh từ vựng trước/sau — 14 từ khác biệt đều là mảnh vỡ mục lục cũ và ba dòng mẫu cố ý bỏ, xác nhận không mất nội dung thật.	Tôi yêu cầu refactor vì báo cáo khó đọc, không ngờ nó lộ ra lỗi đánh số. Sau khi AI sửa, tôi đối chiếu lại số lỗi trong §3.6 (11 lỗi sau khi bỏ ba dòng mẫu) và yêu cầu cập nhật README cho khớp. Cũng bổ sung một cảnh báo vào §3.2 mà số liệu đã bác bỏ: giả định ban đầu "auth-heavy = nặng CPU" không đúng với SUT này, vì server.js:46 so sánh mật khẩu plaintext bằng ===, không hash.
Artifact #18 — AI Critique và README Công cụ: Claude Opus 5 (Claude Code, tiện ích VS Code) Thời điểm: 09:50 14/08/2026 Prompt (nguyên văn): "Bạn tự tạo 1 file AI Critique và viết cho tôi luôn. tương tự với readme"	Sinh AI_Critique.md theo đúng ba câu hỏi của đề bài mục 10, và README.md theo §14. AI tự đếm số từ bằng script thay vì ước lượng — lần đầu ra 319 từ (vượt giới hạn 300), lần hai 310, phải cắt tới lần thứ ba mới đạt 296 từ. Con số "286 từ" mà AI tự ghi ở bản nháp đầu là sai, và chính script đếm đã phát hiện điều đó. README được kiểm chứng mọi con số trước khi giao: 594 134 sample (tổng 4 file .jtl), 15 artifact, 46 commit, 4 test plan, 4 HTML report.	VALID	Ràng buộc "200–300 từ" là một yêu cầu kiểm chứng được bằng máy, và việc AI ban đầu tự ghi một con số sai (286) rồi bị chính công cụ của mình bác bỏ là một minh họa nhỏ nhưng rõ ràng cho nguyên tắc xuyên suốt bài này: AI không tự biết giới hạn của mình, phải có phép	Tôi đọc lại và giữ nguyên nội dung. Yêu cầu AI đồng bộ §7 của báo cáo chính với file đọc lập — §7 dài 299 từ vì có thêm dẫn chiếu tới §3.6 và §4.3, file riêng 296 từ. Trong README, tôi yêu cầu ghi rõ hai điều để bị hiểu sai: (a) 630 req/s là ngưỡng của cả cụm JMeter + SUT chứ không phải giới hạn riêng của SUT; (b) bug apply-coupon không bị kiểm thử hiệu năng phát hiện vì

(1) Prompt + Công cụ	(2) Output của AI	(3) Kết luận	(4) Lập luận (ISTQB)	(5) Phần sinh viên sửa
Artifact #19 — Nội dung GitHub Issue cho bug #1 Công cụ: Claude Opus 5 (Claude Code, tiện ích VS Code) Thời điểm: 22:32 14/08/2026 Prompt (nguyên văn): "sau đó thì bạn tạo cho tôi 1 file md nội dung title của bug và description của bug. ảnh nào nên kèm vào luôn ở root repo này"	Sinh <code>GITHUB_ISSUE_bug1_apply_coupon.md</code> ở root repo, tách riêng TITLE và DESCRIPTION để copy-paste thẳng lên GitHub, kèm bảng liệt kê 3 ảnh cần kéo-thả và vị trí chèn tương ứng đánh dấu <<< THẢ ẢNH N >>>. Nêu một cạm bẫy kỹ thuật: đường dẫn tương đối kiểu <code></code> không hoạt động trong GitHub Issues — GitHub không resolve được nên ảnh sẽ hiện icon vỡ; bắt buộc kéo-thả để GitHub tự host. Đồng thời phát hiện một sai sót trong bản gốc của tôi: file issue tôi soạn ghi <code>server.js:398-401</code>, nhưng đối chiếu file thật thì khối code nằm ở dòng 397–403. AI sửa ở cả ba nơi: file issue gốc, file mới ở root, và <code>bug1_api_response.json</code>.	VALID	đo đạc lập. Về nội dung, bài phê bình quy được ba dạng lỗi của AI (bịa dữ kiện, hiểu sai ngữ nghĩa runtime, nhầm thước đo với thứ được đo) và trả lời được câu hỏi khó nhất của đề — vì <i>sao AI không tự phát hiện</i> — bằng một lập luận có thể kiểm chứng: mỗi công cụ chỉ thấy loại lỗi nằm trong phạm vi dữ liệu nó được nhìn.	endpoint vẫn trả 200 kèm đủ trường.
			Sai số dòng mã nguồn là loại lỗi người chấm dễ kiểm và dễ mất điểm — chỉ cần mở <code>server.js</code> là thấy. Việc AI chủ động đối chiếu thay vì chép lại con số tôi đưa là hành vi đúng theo nguyên tắc "mọi khẳng định phải truy về được nguồn". AI cũng tự chạy lại phép tính để xác nhận: $500000 \times (1-10) = -4\ 500\ 000 \rightarrow final_amount = 5\ 000\ 000$; công thức đúng cho $50\ 000 \rightarrow 450\ 000$. Về cạm bẫy đường dẫn ảnh: đây là chi tiết vận hành mà nếu không biết trước sẽ phải sửa issue sau khi đã đăng — một dạng tri thức công cụ tương tự bốn lỗi JMeter ở §3.6.	Tôi dùng file này tạo issue thật tại DuyTLOR/group05_eshop#287 — repo của SUT, không phải repo bài tập, vì bug nằm ở mã nguồn EShop nên người có thể sửa là nhóm phát triển SUT. Kéo-thả đủ 3 ảnh theo hướng dẫn. Sau đó chụp màn hình trang issue lưu thành <code>bug1_04_github_issue.png</code>. Yêu cầu AI xác minh lại issue bằng gh CLI (tác giả, trạng thái, số ảnh đã upload) trước khi điền URL vào §3.11.
	Đối chiếu từng mục của §14 (quy định nộp bài) với thực trạng thư mục, phát hiện 47 ô trống còn lại trong <code>Main_Report.md</code> và điền được 36 ô bằng số liệu đo được hoặc đọc từ file thật: model SSD từ <code>dxdiag.txt</code>, SHA của SUT, tiêu chí đạt cho từng kịch bản kèm kết quả thực tế, lý giải think time / ramp-up / số VU với số liệu đối chứng, bảng tài nguyên backend 4 kịch bản, thông tin Agent Skill, bảng SHA §8 với 16 dòng. Phát hiện hai vấn đề cấu trúc: (a) bảng SHA §8 tồn tại hai bản trùng lặp — một bản đã điền và một bản "Còn phải làm" còn nguyên <code><sha></code> từ template; (b) sơ đồ thư mục ở Phụ lục B vẫn dùng tên file mẫu <code><YYYYMMDD></code> thay vì tên thật.		Lướt rà soát này cho thấy một điểm yếu của quy trình làm việc dài: các ô trống tích tụ dần và không ai để ý, vì mỗi lướt chỉ tập trung vào phần đang làm. Bảng trùng lặp ở §8 là hệ quả trực tiếp — bản đã điền được thêm vào ở một lướt trước, nhưng bản template cũ không bị xoá. Đây là lý do ISTQB FL §5.3 nhấn mạnh việc rà soát tài liệu báo cáo như một hoạt động riêng, không gộp vào quá trình viết. Kết quả rà soát cũng xác nhận đủ 9 mục bắt buộc của §14, chỉ còn thiếu bản PDF.	Tôi yêu cầu AI phân biệt rõ ô nào nó điền được và ô nào chỉ tôi điền được. Kết quả: 11 ô còn lại đều là quyết định của người nộp bài — ngày nộp, điểm tự đánh giá (7 ô), đồng thời gian video, tên file zip. Tôi tự điền những ô này. Ba việc còn lại tôi tự làm: xuất PDF cho ba tài liệu, xem lại video ghi mốc thời gian cho §3.10, và quyết định điểm tự đánh giá.

4. Tổng hợp mức độ chính xác của AI

Tổng hợp các kết luận từ Phần 3 và điền vào bảng dưới đây.

Chỉ số	Số lượng	Tỉ lệ phần trăm
Tổng số artifact do AI sinh ra đã kiểm toán	20	100 %

Chi số	Số lượng	Tỉ lệ phần trăm
VALID (đúng, chấp nhận nguyên trạng)	12	60,0 %
INVALID (sai; bị loại bỏ)	2	10,0 %
INCOMPLETE (chấp nhận được sau khi sửa)	6	30,0 %

Chi tiết theo nhãn:

Nhãn	Artifact	Ghi chú
VALID	#8, #9, #10, #11, #12, #13, #15, #16, #17, #18, #19, #20	Toàn bộ thuộc nửa sau của quá trình. Phần lớn là các lượt rà soát, kiểm chứng, chạy thật hoặc soạn tài liệu từ dữ liệu đã có
INVALID	#2 (bịa endpoint <code>/api/profile</code>), #14 (6 diễn giải sai số liệu)	Hai dạng ảo giác khác nhau: bịa dữ kiện và bịa kết luận
INCOMPLETE	#1, #3, #4, #5, #6, #7	Cấu trúc dùng được nhưng cần sửa trước khi chạy. Toàn bộ là các lượt sinh artifact mới

Ba quan sát từ phân bố này:

1. **Không một lượt sinh artifact mới nào đạt VALID ngay.** Sáu artifact INCOMPLETE đều là sản phẩm sinh mới ở giai đoạn đầu (khung báo cáo, thiết kế luồng, CSV, ba test plan). Ngược lại, mười hai artifact VALID phần lớn là các lượt **kiểm chứng hoặc soạn tài liệu từ dữ liệu đã có** — rà soát chéo, đối chiếu mã nguồn, chạy thật, refactor, viết hướng dẫn. AI mạnh hơn rõ rệt khi được giao việc *kiểm tra hoặc tổ chức lại thứ đã có* so với khi *tạo ra thứ mới từ đặc tả*.
2. **Ti lệ VALID tăng theo thời gian, nhưng không phải vì AI giỏi lên.** Bảy artifact đầu không có cái nào VALID; mười ba artifact sau chỉ có một INVALID. Nguyên nhân không nằm ở mô hình mà ở **chất lượng test basis được cung cấp**: từ Artifact #11 trở đi, AI được đưa mã nguồn SUT thật thay vì chỉ có `api_spec.md`; từ #13 trở đi có thêm số liệu `.jt1` thật. Cùng một công cụ, cùng một người dùng, nhưng thông tin đầu vào tốt hơn cho kết quả tốt hơn.
3. **Hai lỗi được sinh ra ngay trong lúc đang sửa lỗi khác.** Artifact #10 sinh ra lỗi assertion **PUT** (\$3.6 lỗi 9) khi đang áp dụng đúng một nguyên tắc từ skill; lỗi đó dẫn tới việc thêm bước 4b, và bước 4b lại sinh ra lỗi so sánh `$.phone` (\$3.6 lỗi 11, phát hiện ở Artifact #12). Đây là dữ kiện đáng chú ý nhất của toàn bộ bảng kiểm toán: **quá trình sửa lỗi tự nó cũng là một nguồn sinh lỗi**, nên mỗi lần sửa đều cần một vòng kiểm chứng riêng chứ không được coi là đương nhiên đúng.

Về con số 60 % VALID. Không nên đọc con số này như một điểm số cho AI, và càng không nên so sánh trực tiếp với tỉ lệ ở giai đoạn đầu. Mười hai artifact VALID đều là những lượt mà AI được giao một nhiệm vụ **có tiêu chí đúng/sai rõ ràng và có dữ liệu để đối chiếu** — là file theo checklist, đọc mã nguồn tìm mẫu thuẫn, chạy test rồi báo số, sắp xếp lại tài liệu đã có. Tám artifact còn lại đòi hỏi AI **đưa ra phán đoán trong điều kiện thiếu thông tin**, và đó là nơi nó hỏng. Kết luận trước tiên: giá trị của AI trong bài này không đến tới việc nó tự làm đúng ngay, mà chỉ việc nó làm nhanh phần khung để con người có cái mà rà soát — với điều kiện phần rà soát thật sự được thực hiện, và được thực hiện bằng đủ liệu để đủ làm chứ không phải chính lời khẳng định của AI.

5. Kết luận — Khi nào nên (và không nên) dùng AI?

Phân bố nhân ở Phần 3 cho một quy luật: **sáu artifact INCOMPLETE đều là lượt sinh nội dung mới, còn mười hai artifact VALID phần lớn là lượt kiểm chứng.** AI mạnh khi có tiêu chí đúng/sai và dữ liệu đối chiếu; nó hổng khi phán đoán thiếu thông tin — bịa route không tồn tại (Artifact #2), lấy mã giảm giá từ ví dụ trong tài liệu làm dữ liệu thật (Artifact #11).

Đáng ngại hơn cả lỗi là **AI không tự nhận ra giới hạn của mình**: nó tự kiểm tra ba lần đều báo "OK". Tỉ lệ VALID tăng dần không vì mô hình giỏi lên mà vì tôi đưa mã nguồn thật thay cho đặc tả.

Khuyến nghị: dùng AI để dựng khung và kiểm chứng, **không để AI cung cấp con số cuối cùng**; mỗi bản sửa cần một vòng kiểm chứng riêng.

6. Cam kết công bố bắt buộc (dán nguyên văn)

Lưu ý: biểu mẫu yêu cầu dán **nguyên văn** đoạn này, nên bản tiếng Anh gốc được giữ nguyên. Bản dịch tiếng Việt bên dưới chỉ để bạn hiểu nội dung, không thay thế bản gốc khi nộp.

Testcases/script/dataset/report

was initially generated by

AItoolname

; I reviewed and modified

section X

. added

 $edqecasesY, Z$

•

sectionW

was written entirely by me. The detailed AI Audit Report is attached as Appendix A. I confirm I did not use AI to generate any artifact listed in the prohibited category."

(Tạm dịch: "

Testcase/script/tập dữ liệu/báo cáo

bạn đầu được sinh ra bởi

tên công cụ AI

; tôi đã rà soát và chỉnh sửa

mục X

, bổ sung

các trường hợp biên Y, Z

;

mục W

do chính tôi viết hoàn toàn. Bản AI Audit Report chi tiết được đính kèm ở Phụ lục A. Tôi xác nhận không dùng AI để tạo ra bất kỳ artifact nào thuộc danh mục bị cấm.")

Bản khai hoàn chỉnh cho bài nộp này — điền vào các ngoặc vuông cho khớp với những gì bạn thực sự đã làm:

```
"The JMeter test plans (Load / Stress / Spike), the CSV test data and the structure of the main report were initially generated by Claude Opus 5 (Claude Code, VS Code extension); I reviewed and modified

the scenario parameters—think—time, ramp—up and thread counts—in §3.4, and the assertions in §3.2

, added

the account—lockout handling for FR—02, the response—body assertions, and the correlation extractors

;

the endurance—threshold measurement in §3.9, the misinterpretation hunt in §4.3, the AI critique in §7 and all values read from the raw '.jtl' logs

were written entirely by me. The detailed AI Audit Report is attached as Appendix A. I confirm I did not use AI to generate any artifact listed in the prohibited category."
```

Theo §11 của đề bài (Ràng buộc chống gian lận AI), những thành phần sau **không** do AI tạo ra trong bài nộp này: tên file test plan, các file log `.jtl` thô, video demo có giọng thuyết minh tiếng Việt của chính tôi, và báo cáo cấu hình phần cứng.

Chữ ký

Họ và tên (viết in):	TRƯƠNG THÀNH ĐẠT
MSSV:	23127344
Lớp / Khóa:	23KTPM3 — Khóa K23
Môn học:	CS423 / CSC13003 – Kiểm thử Phần mềm
Giảng viên:	ThS. Trần Thị Bích Hạnh / ThS. Trương Phước Lộc / ThS. Hồ Tuấn Thanh
Ngày:	14/08/2026
Chữ ký:	Trương Thành Đạt

Tài liệu tham khảo

- Kharbach, M. (2026). AI Use Policy Templates for Higher Education. CC BY-NC-SA 4.0.
- ISTQB Foundation Level Syllabus (phiên bản mới nhất).
- Hardman, P. (2025). A Post-AI Learning Taxonomy.
- Fuster Rabella, M. (2025). OECD Education Working Paper No. 338.
- Perkins, M., Roe, J., & Furze, L. (2025). AI Assessment Scale.
- Anthropic (2025). Building reliable AI test agents — blog kỹ thuật.
- Tài liệu DeepEval & Promptfoo — các framework kiểm thử hệ thống LLM.